

Processes

Practice Exercises

- 3.1 Using the program shown in Figure 3.30, explain what the output will be at LINE A.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0) { /* child process */
        value += 15;
        return 0;
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINE A */
        return 0;
    }
}
```

Figure 3.30 What output will be at Line A?

Answer:

The result is still 5, as the child updates its copy of the value. When control returns to the parent, its value remains at 5.

```

#include <stdio.h>
#include <unistd.h>

int main()
{
    /* fork a child process */
    fork();

    /* fork another child process */
    fork();

    /* and fork another */
    fork();

    return 0;
}

```

Figure 3.31 How many processes are created?

- 3.2 Including the initial parent process, how many processes are created by the program shown in Figure 3.31?

Answer:

Eight processes are created.

- 3.3 Original versions of Apple's mobile iOS operating system provided no means of concurrent processing. Discuss three major complications that concurrent processing adds to an operating system.

Answer:

1. Difficulty in sharing global resources
 2. Difficulty in resource allocation
 3. Issues with debugging and programming errors
- a. The CPU scheduler must be aware of the different concurrent processes and must choose an appropriate algorithm that schedules the concurrent processes.
 - b. Concurrent processes may need to communicate with one another, and the operating system must therefore develop one or more methods for providing interprocess communication.
 - c. Because mobile devices often have limited memory, a process that manages memory poorly will have an overall negative impact on other concurrent processes. The operating system must therefore manage memory to support multiple concurrent processes.
- 3.4 Some computer systems provide multiple register sets. Describe what happens when a context switch occurs if the new context is already loaded into one of the register sets. What happens if the new context is in memory rather than in a register set and all the register sets are in use?

If the new context is already loaded into one of the register sets, then the CPU simply changes the pointer to the current register set to point to the set containing the new context. This is a very fast operation, as it only requires changing a few bits in the CPU's registers.

If the new context is in memory rather than in a register set, and all the register sets are in use, then one of the contexts in a register set must be chosen and be moved to memory, and the new context must be loaded from memory into the set. This operation takes a little more time than on systems with new context is already on one of the register set, as it requires copying the register data to and from memory.

Answer:

The CPU current-register-set pointer is changed to point to the set containing the new context, which takes very little time. If the context is in memory, one of the contexts in a register set must be chosen and moved to memory, and the new context must be loaded from memory into the set. This process takes a little more time than on systems with one set of registers, depending on how a replacement victim is selected.

- 3.5 When a process creates a new process using the `fork()` operation, which of the following states is shared between the parent process and the child process?

- a. Stack
- b. Heap
- c. Shared memory segments

There are a number of ways to implement "exactly once" semantics in RPC. One common approach is to use a combination of acknowledgments (ACKs) and timestamps. When a client sends an RPC to a server, it also includes a timestamp. The server then acknowledges the receipt of the RPC by sending back an ACK with a timestamp. If the client does not receive an ACK within a certain time period, it can assume that the RPC was not received by the server and resend it.

Answer:

Only the shared memory segments are shared between the parent process and the newly forked child process. Copies of the stack and the heap are made for the newly created process.

- 3.6 Consider the "exactly once" semantic with respect to the RPC mechanism. Does the algorithm for implementing this semantic execute correctly even if the ACK message sent back to the client is lost due to a network problem? Describe the sequence of messages, and discuss whether "exactly once" is still preserved.

Answer:

The "exactly once" semantics ensure that a remote procedure will be executed exactly once and only once. The general algorithm for ensuring this combines an acknowledgment (ACK) scheme combined with timestamps (or some other incremental counter that allows the server to distinguish between duplicate messages).

The general strategy is for the client to send the RPC to the server along with a timestamp. The client will also start a timeout clock. The client will then wait for one of two occurrences: (1) it will receive an ACK from the server indicating that the remote procedure was performed, or (2) it will time out. If the client times out, it assumes the server was unable to perform the remote procedure, so the client invokes the RPC a second time, sending a later timestamp. The client may not receive the ACK for one of two reasons: (1) the original RPC was never received by the server, or (2) the RPC was correctly received—and performed—by the server but the ACK was lost. In situation (1), the use of ACKs allows the server ultimately to receive and perform the RPC. In situation (2), the server will receive a duplicate RPC, and it will use the timestamp to identify it as a duplicate so as not to perform the RPC a second time. It is important to note that the server must send a second ACK back to the client to inform the client the RPC has been performed.

In an RPC system, a client process sends a request message to a remote server process, which executes the requested procedure and sends a response message back to the client. The client and server processes can be running on different computers, and the communication between them is transparent to the client process.

- 3.7 Assume that a distributed system is susceptible to server failure. What mechanisms would be required to guarantee the “exactly once” semantic for the execution of RPCs?

Answer:

The server should keep track of stable storage (such as a disk log) of information regarding what RPC operations were received, whether they were successfully performed, and the results associated with the operations. When a server crash takes place and an RPC message is received, the server can check whether the RPC has been previously performed and therefore guarantee “exactly once” semantics for the execution of RPCs.

10E Exercises

- 3.8 Describe the actions taken by a kernel to context-switch between processes.

Answer:

In general, the operating system must save the state of the currently running process and restore the state of the process scheduled to be run next. Saving the state of a process typically includes the values of all the CPU registers in addition to memory allocation. Context switches must also perform many architecture-specific operations, including flushing data and instruction caches.

In an operating system, a context switch is the process of saving the current state of a running process or thread, and restoring the saved state of another process or thread to allow it to continue execution. The context of a process or thread includes information such as the values in its registers, its program counter, and its stack pointer.

- 3.9 Construct a process tree similar to Figure 3.7. To obtain process information for the UNIX or Linux system, use the command `ps -aef`. Use the command `man ps` to get more information about the `ps` command. The task manager on Windows systems does not provide the parent process ID, but the process monitor tool, available from technet.microsoft.com, provides a process-tree tool.

Answer:

Results will vary widely.

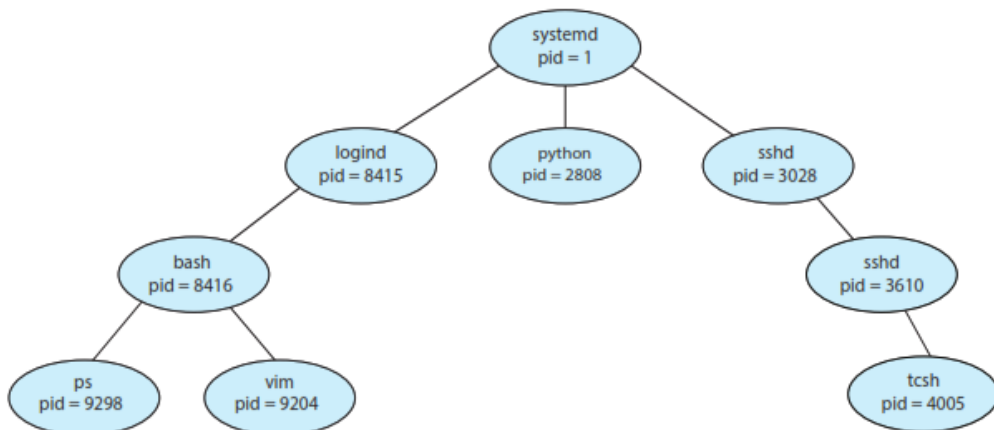


Figure 3.7 A tree of processes on a typical Linux system.

A zombie process in an operating system is a process that has completed its execution, but its entry in the process table is not yet removed. This occurs because the parent process has not yet called the `wait()` system call to reap the zombie process.

- 3.10 Explain the role of the `init` (or `systemd`) process on UNIX and Linux systems in regard to process termination.

Answer:

When a process is terminated, it briefly moves to the zombie state and remains in that state until the parent invokes a call to `wait()`. When this occurs, the process id, as well as the entry in the process table, are both released. However, if a parent does not invoke `wait()`, the child process remains a zombie as long as the parent remains alive. Once the parent process terminates, the `init` process becomes the new parent of the zombie. Periodically, the `init` process calls `wait()` which ultimately releases the pid and entry in the process table of the zombie process.

Traditional UNIX systems identify the process `init` as the root of all child processes. `init` (also known as System V `init`) is assigned a pid of 1, and is the first process created when the system is booted. On a process tree, `init` is at the root.

Linux systems initially adopted the System V `init` approach, but recent distributions have replaced it with `systemd`. `Systemd` serves as the system's initial process, much the same as System V `init`; however, it is much more flexible and can provide more services, than `init`.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;

    for (i = 0; i < 4; i++)
        fork();

    return 0;
}
```

Figure 3.32 How many processes are created?

- 3.11 Including the initial parent process, how many processes are created by the program shown in Figure 3.32?

Answer:

Eight processes are created.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
```

```

int main()
{
    pid_t pid;
    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
        printf("LINE J");
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}

```

Figure 3.33 When will LINE J be reached?

- 3.12 Explain the circumstances under which the line of code marked `printf("LINE J")` in Figure 3.33 will be reached.

Answer:

The call to `exec()` replaces the address space of the process with the program specified as the parameter to `exec()`. If the call to `exec()` succeeds, the new program is now running and control from the call to `exec()` never returns. In this scenario, the line `printf("Line J");` would never be performed. However, if an error occurs in the call to `exec()`, the function returns control and therefore the line `printf("Line J");` would be performed.

```

#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid, pid1;
    /* fork a child process */
    pid = fork();

```

```

        if (pid < 0) { /* error occurred */
            fprintf(stderr, "Fork Failed");
            return 1;
        }
        else if (pid == 0) { /* child process */
            pid1 = getpid();
            printf("child: pid = %d", pid); /* A */
            printf("child: pid1 = %d", pid1); /* B */
        }
        else { /* parent process */
            pid1 = getpid();
            printf("parent: pid = %d", pid); /* C */
            printf("parent: pid1 = %d", pid1); /* D */
            wait(NULL);
        }

        return 0;
    }
}

```

Figure 3.34 What are the pid values?

- 3.13 Using the program in Figure 3.34, identify the values of pid at lines A, B, C, and D. (Assume that the actual pids of the parent and child are 2600 and 2603, respectively.)

Answer:

A = 0, B = 2603, C = 2603, D = 2600

- 3.14 Give an example of a situation in which ordinary pipes are more suitable than named pipes and an example of a situation in which named pipes are more suitable than ordinary pipes.

Answer:

Simple communication works well with ordinary pipes. For example, assume we have a process that counts characters in a file. An ordinary pipe can be used where the producer writes the file to the pipe and the consumer reads the files and counts the number of characters in the file. Next, for an example where named pipes are more suitable, consider the situation where several processes may write messages to a log. When processes wish to write a message to the log, they write it to the named pipe. A server reads the messages from the named pipe and writes them to the log file.

- 3.15 Consider the RPC mechanism. Describe the undesirable consequences that could arise from not enforcing either the “at most once” or “exactly once” semantic. Describe possible uses for a mechanism that has neither of these guarantees.

Answer:

If an RPC mechanism cannot support either the “at most once” or “at least once” semantics, then the RPC server cannot guarantee that a remote procedure will not be invoked multiple occurrences. Consider if a

remote procedure was withdrawing money from a bank account on a system that did not support these semantics. It is possible that a single invocation of the remote procedure might lead to multiple withdrawals on the server.

For a system to support either of these semantics generally requires the server to maintain some form of client state such as the timestamp described in the text.

If a system were unable to support either of these semantics, then such a system could only safely provide remote procedures that do not alter data or provide time-sensitive results. Using our bank account as an example, we certainly require “at most once” or “at least once” semantics for performing a withdrawal (or deposit!). However, an inquiry into an account balance or other account information such as name, address, etc. does not require these semantics.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE 5

int nums[SIZE] = {0,1,2,3,4};

int main()
{
    int i;
    pid_t pid;
    pid = fork();

    if (pid == 0) {
        for (i = 0; i < SIZE; i++) {
            nums[i] *= -i;
            printf("CHILD: %d ",nums[i]); /* LINE X */
        }
    }
    else if (pid >0) {
        wait(NULL);
        for (i = 0; i < SIZE; i++)
            printf("PARENT: %d ",nums[i]); /* LINE Y */
    }

    return 0;
}
```

Figure 3.35 What output will be at Line X and Line Y?

- 3.16 Using the program shown in Figure 3.35, explain what the output will be at lines X and Y.

Answer:

Because the child is a copy of the parent, any changes the child makes will occur in its copy of the data and won't be reflected in the parent. As a result, the values output by the child at line X are 0, -1, -4, -9, and -16. The values output by the parent at line Y is 0, 1, 2, 3, and 4.

- 3.17 What are the benefits and disadvantages of each of the following? Consider both the system level and the programmer level.

- a. Synchronous and asynchronous communication
- b. Automatic and explicit buffering
- c. Send by copy and send by reference
- d. Fixed-sized and variable-sized messages

Answer:

- a. **Synchronous and asynchronous communication** — A benefit of synchronous communication is that it allows a rendezvous between the sender and receiver. A disadvantage of a blocking send is that a rendezvous may not be required and the message could be delivered asynchronously. As a result, message-passing systems often provide both forms of synchronization.
- b. **Automatic and explicit buffering** — Automatic buffering provides a queue with indefinite length, thus ensuring the sender will never have to block while waiting to copy a message. There are no specifications on how automatic buffering will be provided; one scheme may reserve sufficiently large memory where much of the memory is wasted. Explicit buffering specifies how large the buffer is. In this situation, the sender may be blocked while waiting for available space in the queue. However, it is less likely that memory will be wasted with explicit buffering.
- c. **Send by copy and send by reference** — Send by copy does not allow the receiver to alter the state of the parameter; send by reference does allow it. A benefit of sending by reference is that it allows the programmer to write a distributed version of a centralized application. Java's RMI provides both; however, passing a parameter by reference requires declaring the parameter as a remote object as well.
- d. **Fixed-sized and variable-sized messages** — The implications of this are mostly related to buffering issues; with fixed-size messages, a buffer with a specific size can hold a known number of messages. The number of variable-sized messages that can be held by such a buffer is unknown. Consider how Windows 2000 handles this situation: with fixed-sized messages (anything < 256 bytes), the messages are copied from the address space of the sender to the address space of the receiving process. Larger messages (i.e. variable-sized messages) use shared memory to pass the message.

9E Exercises

- 3.8 Describe the differences among short-term, medium-term, and long-term scheduling.

Answer:

- a. **Short-term** (CPU scheduler)— selects from jobs in memory those jobs that are ready to execute and allocates the CPU to them.
- b. **Medium-term** — used especially with time-sharing systems as an intermediate scheduling level. A swapping scheme is implemented to remove partially run programs from memory and reinstate them later to continue where they left off.
- c. **Long-term** (job scheduler) — determines which jobs are brought into memory for processing.

The primary difference is in the frequency of their execution. The short-term must select a new process quite often. Long-term is used much less often since it handles placing jobs in the system and may wait a while for a job to finish before it admits another one.

Solution by D. M. Nadim Hayder